

CRUD + EXERCÍCIOS

Deixando aqui para facilitar a vida de vocês:

CREATE:

```
-- CREATE: Insere novos registros (linhas) em uma tabela -  
  
INSERT INTO nome_da_tabela (coluna1, coluna2)  
VALUES ('valor1', 'valor2');
```

READ:

```
-- READ: Recupera e consulta dados já existentes no banco. --  
  
SELECT coluna1, coluna2  
FROM nome_da_tabela  
WHERE condicao;
```

UPDATE:

```
-- UPDATE: Modifica os dados de registros que já existem na tabela. --  
  
UPDATE nome_da_tabela  
SET coluna1 = 'novo_valor'  
WHERE condicao;
```

DELETE:

```
-- DELETE: Remove registros do banco de dados.Aviso: Nunca esqueça a cláusula WHERE, caso contrário, todos os dados da tabela serão apagados. --
```

```
DELETE FROM nome_da_tabela  
WHERE condicao;
```

Por favor, não apague o banco de dados da empresa de vocês, senão vocês estarão no BH no dia seguinte sendo demitidos kkk |

EXPLICAÇÃO

Os exercícios focam na fixação da sintaxe básica e estrutural das operações para estimular o raciocínio lógico.

Vamos evoluir um pouco e utilizar operadores relacionais mais avançados em BD: (IN, BETWEEN, LIKE), funções de agregação (COUNT, AVG), ordenação múltipla (ORDER BY), manipulação de valores nulos (IS NULL) e atualizações dinâmicas.

1. OPERADORES RELACIONAIS E LÓGICOS

IN

O operador IN substitui o uso repetitivo do operador OR. Ele verifica se um valor específico corresponde a qualquer valor dentro de uma lista definida por você. É excelente para filtrar categorias ou IDs específicos.

Exemplo: Buscar alunos que estão nos cursos de Matemática, Física ou Química.

```
SELECT nome, curso FROM alunos  
WHERE curso IN ('Matemática', 'Física', 'Química');
```

BETWEEN

O operador BETWEEN seleciona valores dentro de um intervalo determinado. Ele funciona com números, textos e datas. Um detalhe importante: o intervalo é **inclusivo**, ou seja, os valores de início e fim que você definir também entram no resultado.

- **Exemplo:** Buscar alunos que tiraram notas entre 6.0 e 8.5 (incluindo o 6.0 e o 8.5).

```
SELECT nome, nota FROM alunos  
WHERE nota BETWEEN 6.0 AND 8.5;
```

LIKE

O LIKE é usado na cláusula WHERE para buscar um padrão específico em uma coluna de texto. Ele é quase sempre acompanhado de "curingas" (wildcards):

- O sinal de porcentagem % representa zero, um ou múltiplos caracteres.
- O sublinhado _ representa um único caractere.
- **Exemplo:** Buscar alunos cujo nome comece com a letra "C" (não importa o que venha depois).

```
SELECT nome FROM alunos  
WHERE nome LIKE 'C%';
```

2. FUNÇÕES DE AGREGAÇÃO

As funções de agregação realizam um cálculo em um conjunto de valores e retornam um único valor.

COUNT()

Como o nome sugere, ele conta o número de linhas (registros) que correspondem a um critério especificado. Pode ser usado para contar tudo (COUNT(*)) ou apenas os registros não nulos de uma coluna específica (COUNT(coluna)).

- **Exemplo:** Descobrir o número total de alunos matriculados no curso de História.

```
SELECT COUNT(*) FROM alunos  
WHERE curso = 'História';
```

AVG()

Vem da palavra *Average* (Média). Ele calcula e retorna o valor médio de uma coluna numérica.

- **Exemplo:** Calcular a média geral de notas de todos os alunos do banco.

```
SELECT AVG(nota) FROM alunos;
```

3. ORDENAÇÃO MÚLTIPLA

ORDER BY

O ORDER BY é usado para classificar o conjunto de resultados em ordem crescente (ASC, que é o padrão) ou decrescente (DESC). A **ordenação múltipla** ocorre quando você passa mais de uma coluna para ele organizar. O banco ordena primeiro pela coluna 1; se houver empate, ele usa a coluna 2 para desempatar, e assim por diante.

- **Exemplo:** Ordenar a lista de alunos primeiro pelo curso (em ordem alfabética) e, dentro de cada curso, colocar os alunos com as maiores notas primeiro.

```
SELECT nome, curso, nota FROM alunos  
ORDER BY curso ASC, nota DESC;
```

4. MANIPULAÇÃO DE VALORES NULOS

IS NULL / IS NOT NULL

Em bancos de dados, NULL não é zero nem um espaço em branco; significa "ausência de valor" (ou valor desconhecido). Por isso, você não pode usar = NULL ou != NULL. Você deve perguntar ao banco se aquele campo *é nulo* ou *não é nulo*.

- **Exemplo:** Encontrar todos os alunos que ainda não receberam nota.

```
SELECT nome, curso FROM alunos  
WHERE nota IS NULL;
```

5. ATUALIZAÇÕES DINÂMICAS

Uma atualização dinâmica acontece quando o comando UPDATE não define um valor fixo e estático (como `nota = 10`), mas sim um valor que **depende dos próprios dados existentes na tabela** ou de regras lógicas.

Isso pode ser feito referenciando a própria coluna matemática:

- **Exemplo:** Aumentar a nota atual de todos os alunos do curso de Física em 1 ponto. (*Se a nota era 6, vira 7. Se era 8, vira 9*).

```
UPDATE alunos
SET nota = nota + 1.0
WHERE curso = 'Física';
```

Também pode envolver lógica condicional dentro do comando usando CASE WHEN, permitindo que um único comando aplique regras diferentes para registros diferentes:

- **Exemplo:** Atualizar o status baseado na nota.

```
UPDATE alunos
SET status_matricula = CASE
    WHEN nota >= 6.0 THEN 'Aprovado'
    ELSE 'Retido'
END;
```

ENUNCIADO

Lembram que fizeram a tabela alunos na aula passada? Então, vocês vão adicionar mais duas colunas:

- **id** (número identificador único)
- **nome** (texto)
- **idade** (número)
- **curso** (texto)
- **nota** (número decimal)
- **data_matricula** (formato de data)
- **status_matricula** (texto, podendo ser 'ativo', 'trancado' ou 'concluido')

Ah, esqueceu quais são as colunas que já existiam? Abram o pdf anterior, ué.

1) 10 CREATES (NÍVEL INTERMEDIÁRIO)

Explorando inserções múltiplas, valores nulos e funções nativas.

1. **Insert múltiplo:** insira três alunos de uma só vez na tabela utilizando apenas um comando INSERT.
2. **Uso de data atual:** Insira um aluno no curso de "Desenvolvimento de Sistemas" utilizando a função nativa do banco (como CURRENT_DATE ou NOW()) para preencher a data_matricula.
3. **Omissão de colunas:** Cadastre a aluna "Camila" informando apenas o nome e o curso. As colunas idade, nota e status devem assumir os valores nulos ou padrão do banco.
4. **Caracteres de escape:** Insira um aluno chamado "D'Angelo" (cuidado com as aspas simples no nome) com nota 8.5.
5. **Cálculo na inserção:** Insira uma aluna de "Lógica de Programação" onde a idade não é informada diretamente, mas sim calculada via SQL (ex: 2026 - 2008).
6. **Inserção com status:** Cadastre o aluno "Marcos", nota 5.0, e defina imediatamente o status_matricula dele como 'Trancado'.
7. **Insert ignorando erros (MySQL):** Utilize o comando INSERT IGNORE para tentar inserir um aluno forçando um id que já existe (ex: id 1) para observar como o banco lida com a restrição.
8. **Matrícula antiga:** Adicione a aluna "Sônia" definindo manualmente a data_matricula para o dia '2024-02-15'.
9. **Cálculo de nota:** Insira um aluno transferido chamado "Beto", onde a nota inserida seja o resultado da expressão matemática $(7.5 + 8.0) / 2$.
10. **Inserção completa dinâmica:** Insira a aluna "Letícia" garantindo que o nome seja inserido em letras maiúsculas utilizando a função UPPER().

2) 10 READS (NÍVEL INTERMEDIÁRIO)

1. *Focando em filtros avançados, buscas parciais e agregação.*
2. **Operador BETWEEN:** Busque todos os alunos que possuem nota entre 6.0 e 8.5.
3. **Operador IN:** Liste os alunos que estão matriculados exclusivamente nos cursos de "Matemática", "Física" ou "Química". // O Jonnathan sente saudades de vocês ❤️
4. **Operador LIKE (Início):** Consulte todos os alunos cujo nome comece com a letra "C".
5. **Operador LIKE (Conteúdo):** Encontre todos os alunos que tenham o sobrenome "Silva" em qualquer parte do nome.
6. **Tratamento de Nulos:** Busque todos os alunos cuja nota não foi lançada ainda (IS NULL).
7. **Agregação Simples:** Utilize o comando COUNT() para descobrir o número total de alunos com o status 'Ativo'.
8. **Ordenação Múltipla:** Liste todos os alunos ordenando primeiro pelo curso (ordem alfabética) e, em seguida, pela nota (da maior para a menor).
9. **Exclusão Lógica:** Busque todos os alunos que **NÃO** estão no curso de "Artes" utilizando o operador lógico de negação.
10. **Média Matemática:** Utilize a função AVG() para descobrir a média geral das notas de todos os alunos cadastrados.
11. **Busca de Distintos:** Liste apenas os nomes dos cursos disponíveis na tabela, sem repetir nenhum nome (SELECT DISTINCT).

3) 10 UPDATES (NÍVEL INTERMEDIÁRIO)

Atualizações dinâmicas, com cálculos e múltiplas condições.

1. **Acréscimo Percentual:** Aumente a nota em 10% ($\text{nota} = \text{nota} * 1.1$) para todos os alunos do curso de "Banco de Dados".
2. **Condição Dupla (AND):** Mude o status_matricula para 'Trancado' de todos os alunos que têm nota menor que 4.0 E possuem mais de 18 anos.
3. **Atualização Múltipla:** Altere o curso para "Desenvolvimento Web" e a nota para 9.0 do aluno que possui o id igual a 12, em um único comando.
4. **Condição Inclusiva (IN):** Atualize a idade de todos os alunos para 20 anos, mas apenas se o ID deles for 2, 5, 7 ou 10.
5. **Limpando Dados:** Remova a nota (transformando em NULL) de todos os alunos que estão com o status 'Trancado'.
6. **Padronização de Texto:** Atualize os nomes de todos os alunos do curso de "História" para que fiquem totalmente em letras minúsculas (LOWER()).
7. **Condição Alternativa (OR):** Subtraia 0.5 da nota de quem está no curso de "Física" OU "Química".
8. **Condição por Data:** Mude o status para 'Concluido' de todos os alunos cuja data_matricula seja anterior a '2025-01-01'.
9. **Concatenação:** Atualize o nome do aluno de id 3 adicionando " (Representante)" ao final do nome dele.
10. **Ajuste Dinâmico:** Se a nota máxima for 10.0, atualize a nota de todos os alunos que tiraram mais de 10.0 (por erro de digitação) para cravados 10.0.

4) 10 DELETES (NADA É TÃO RUIM QUE NÃO POSSA PIORAR, CALMA)

Remoções seguras exigindo cruzamento rigoroso de regras.

1. **Delete Parcial (LIKE):** Delete todos os alunos cujos nomes terminem com a palavra "Junior".
2. **Delete com IN:** Remova os registros dos alunos que possuem os IDs 4, 8 e 15 em uma única query.
3. **Limpeza de Nulos:** Deletar todos os alunos que não possuem curso atribuído (IS NULL).
4. **Delete por Intervalo:** Exclua do banco os alunos cuja idade esteja entre 30 e 40 anos.
5. **Combinação Exata:** Limpe do sistema todos os alunos do curso de "Geografia" que também estejam com o status 'Trancado'.
6. **Ordenação com Delete (MySQL):** Remova os 3 alunos com as menores notas da escola utilizando ORDER BY e LIMIT junto com o DELETE.
7. **Delete por Exceção:** Exclua todos os registros onde a nota for menor que 5, **EXCETO** se o aluno for do curso de "Desenvolvimento de Sistemas".
8. **Limpeza por Padrão:** Exclua qualquer aluno cujo nome contenha a palavra "Teste".
9. **Validação Rigorosa:** Delete os alunos que têm idade menor que 16 **E** cuja data de matrícula seja do ano vigente.
10. **Remoção Condicional Múltipla:** Deletar todos os registros onde a nota for 0.0 **OU** o status da matrícula for nulo.

DESAFIOS (4 INTERMEDIÁRIOS)

Desafio 1: o raio-x do conselho de classe (leitura com agrupamento)

- **Situação:** A coordenação quer saber quais cursos estão com um bom desempenho geral.
- **O que fazer:** Escreva um comando que mostre o nome do curso e a média de notas de seus alunos, mas **agrupando** os resultados por curso (GROUP BY). Mostre apenas os cursos cuja média final seja maior que 7.0 (Dica: uso do HAVING).

Desafio 2: a migração de sistema (criação dinâmica)

- **Situação:** A Etec vai adotar um novo portal de atendimento ao aluno e precisa exportar os aprovados.
- **O que fazer:** Crie uma nova tabela chamada alunos_aprovados e, no mesmo comando, insira nela todos os dados dos alunos da tabela original que possuem nota maior ou igual a 6.0. (Dica: Pesquise sobre CREATE TABLE AS SELECT).

Desafio 3: a regra de negócio automatizada (update condicional)

- **Situação:** Ficar alterando o status aluno por aluno dá muito trabalho. A secretaria quer rodar um comando único no fim do semestre.
- **O que fazer:** Escreva um comando UPDATE que atualize a coluna status_matricula. Se a nota for maior ou igual a 6.0, o status vira 'Concluido'. Se for menor que 6.0, o status vira 'Retido'. (Dica: Será necessário usar a estrutura CASE WHEN dentro do UPDATE).

Desafio 4: a limpeza de inconsistências (delete com subquery lógica)

- **Situação:** Um bug no sistema duplicou alguns cadastros, mas você sabe que alunos reais sempre têm pelo menos a nota ou o status preenchidos.
- **O que fazer:** Exclua todos os alunos que possuam nota nula **E** que o curso seja o mesmo de um aluno específico, digamos, o "João". (Dica: O WHERE precisará avaliar o curso através de um sub-SELECT buscando o curso do João).

(Não adianta nada vocês pedirem desafios e depois só jogar no chatgpt né? Usem o cérebro)

ASSIM QUE TERMINAREM, ME MOSTREM OS COMANDOS NO SQL COM UM COMENTÁRIO AO FINAL DE CADA BLOCO (CREATE, READ, UPDATE, DELETE) COM O NOME DO ALUNO, DATA DE HOJE, NOME DA MATÉRIA E NOME DA ETEC.